```
Network Anomaly Detection — End-to-End Project

Author: Deepak SVG
Dataset: NSL-KDD style network connection logs (41 features + label)
Goal: Build a binary classifier that flags anomalous network
connections in real time.
Target metric: F1-score (primary) and ROC-AUC (secondary) — we care
equally about catching anomalies (recall) and not crying wolf
(precision).

Notebook structure
Setup & data ingestion
Exploratory Data Analysis (EDA)
Hypothesis testing (5 tests)
Feature engineering & preprocessing
Model training & cross-validation (Logistic Regression, Decision Tree,
Random Forest, XGBoost)
Model evaluation & comparison
Feature importance
Persist artifacts for Flask deployment
Insights & recommendations
1. Setup & data ingestion¶
```

```
import sys
print(sys.executable)
```

```
C:\Users\deepa\anaconda3\envs\mlclean\python.exe
```

```
import numpy as np
import pandas as pd
print(np.__version__)
print(pd.__version__)
```

```
2.4.4
3.0.2
```

```
import os, json, warnings, joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

from sklearn.model_selection import train_test_split, StratifiedKFold,
cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score,
recall_score, f1_score,
```

```
                                roc_auc_score, classification_report,
confusion_matrix, roc_curve)

try:
    from xgboost import XGBClassifier
    HAS_XGB = True
except ImportError:
    HAS_XGB = False
    print('xgboost not installed — install with: pip install xgboost')

warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 5)
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

print('Setup complete.')
```

```
Setup complete.
```

```
# NSL-KDD column names — 41 features + target ('attack') + difficulty
('level' if KDDTrain+ used)
COLUMNS = [

'duration','protocol_type','service','flag','src_bytes','dst_bytes','l
and','wrong_fragment',

'urgent','hot','num_failed_logins','logged_in','num_compromised','root
_shell','su_attempted',

'num_root','num_file_creations','num_shells','num_access_files','num_o
utbound_cmds',

'is_host_login','is_guest_login','count','srv_count','serror_rate','sr
v_serror_rate',

'rerror_rate','srv_rerror_rate','same_srv_rate','diff_srv_rate','srv_d
iff_host_rate',

'dst_host_count','dst_host_srv_count','dst_host_same_srv_rate','dst_ho
st_diff_srv_rate',

'dst_host_same_src_port_rate','dst_host_srv_diff_host_rate','dst_host_
serror_rate',

'dst_host_srv_serror_rate','dst_host_rerror_rate','dst_host_srv_rerror
_rate',
    'attack','level'
]
```

```
DATA_PATH = r"C:\Users\deepa\Documents\Documents\Python_Scalar\
Network_anomaly_data.csv"  # place the downloaded NSL-KDD CSV here

# Normalise camelCase column names (as in the provided CSV) to
snake_case
RENAME_MAP = {

'protocoltype':'protocol_type','srcbytes':'src_bytes','dstbytes':'dst_
bytes',

'wrongfragment':'wrong_fragment','numfailedlogins':'num_failed_logins'
,

'loggedin':'logged_in','numcompromised':'num_compromised','rootshell':
'root_shell',

'suattempted':'su_attempted','numroot':'num_root','numfilecreations':'
num_file_creations',
    'numshells':'num_shells','numaccessfiles':'num_access_files',

'numoutboundcmds':'num_outbound_cmds','ishostlogin':'is_host_login',

'isguestlogin':'is_guest_login','srvcount':'srv_count','serrorrate':'s
error_rate',
    'srvserrorrate':'srv_serror_rate','rerrorrate':'rerror_rate',
    'srvrerrorrate':'srv_rerror_rate','samesrvrate':'same_srv_rate',

'diffsrvrate':'diff_srv_rate','srvdiffhostrate':'srv_diff_host_rate',

'dsthostcount':'dst_host_count','dsthostsrvcount':'dst_host_srv_count'
,

'dsthostsamesrvrate':'dst_host_same_srv_rate','dsthostdiffsrvrate':'ds
t_host_diff_srv_rate',
    'dsthostsamesrcportrate':'dst_host_same_src_port_rate',
    'dsthostsrvdiffhostrate':'dst_host_srv_diff_host_rate',
    'dsthostserrorrate':'dst_host_serror_rate',
    'dsthostsrvserrorrate':'dst_host_srv_serror_rate',
    'dsthostrerrorrate':'dst_host_rerror_rate',

'dsthostsrvrerrorrate':'dst_host_srv_rerror_rate','lastflag':'level',
}

if os.path.exists(DATA_PATH):
    df = pd.read_csv(DATA_PATH)
    # If file has no header, re-read with column names
    if df.shape[1] in (42, 43) and df.columns[0] != 'duration':
        df = pd.read_csv(DATA_PATH, header=None,
names=COLUMNS[:df.shape[1]])
```

```
    df = df.rename(columns=RENAME_MAP)
    df = df.rename(columns=RENAME_MAP)
    print(f'Loaded {df.shape[0]:,} rows × {df.shape[1]} columns from
{DATA_PATH}')
else:
    raise FileNotFoundError(
        f'Dataset not found at {DATA_PATH}. Download from the Drive
link in the brief and place it there.'
    )

df.head()
```

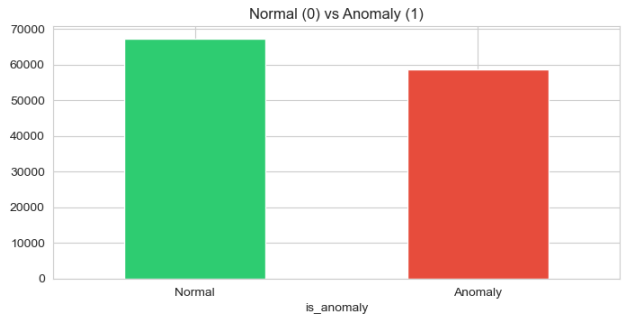
```
Loaded 125,973 rows × 43 columns from C:\Users\deepa\Documents\
Documents\Python_Scalar\Network_anomaly_data.csv

   duration protocol_type   service flag  src_bytes  dst_bytes
land  \
0         0           tcp  ftp_data   SF        491          0     0

1         0           udp     other   SF        146          0     0

2         0           tcp   private   S0          0          0     0

3         0           tcp      http   SF        232       8153     0

4         0           tcp      http   SF        199        420     0


   wrong_fragment  urgent  hot  ...  dst_host_same_srv_rate  \
0               0       0    0  ...                    0.17
1               0       0    0  ...                    0.00
2               0       0    0  ...                    0.10
3               0       0    0  ...                    1.00
4               0       0    0  ...                    1.00

   dst_host_diff_srv_rate  dst_host_same_src_port_rate  \
0                    0.03                         0.17
1                    0.60                         0.88
2                    0.05                         0.00
3                    0.00                         0.03
4                    0.00                         0.00

   dst_host_srv_diff_host_rate  dst_host_serror_rate  \
0                         0.00                  0.00
1                         0.00                  0.00
2                         0.00                  1.00
3                         0.04                  0.03
4                         0.00                  0.00

   dst_host_srv_serror_rate  dst_host_rerror_rate
dst_host_srv_rerror_rate  \
```

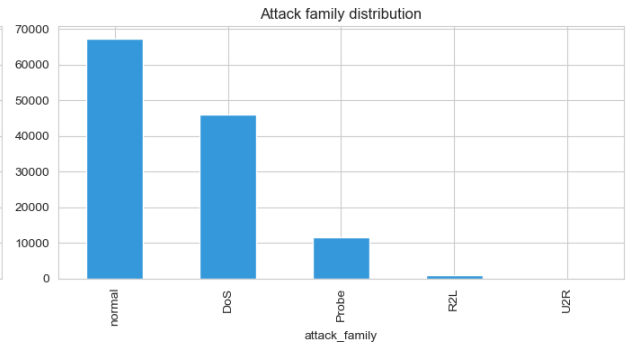
```
0                       0.00                    0.05
0.00
1                       0.00                    0.00
0.00
2                       1.00                    0.00
0.00
3                       0.01                    0.00
0.01
4                       0.00                    0.00
0.00

    attack  level
0   normal     20
1   normal     15
2  neptune     19
3   normal     21
4   normal     21

[5 rows x 43 columns]
```

```
print('Shape:', df.shape)
print('\nDtypes:')
print(df.dtypes.value_counts())
print('\nMissing values per column (top 10):')
print(df.isnull().sum().sort_values(ascending=False).head(10))
print('\nDuplicates:', df.duplicated().sum())
```

```
Shape: (125973, 43)

Dtypes:
int64      24
float64    15
str         4
Name: count, dtype: int64

Missing values per column (top 10):
duration          0
protocol_type     0
service           0
flag              0
src_bytes         0
dst_bytes         0
land              0
wrong_fragment    0
urgent            0
hot               0
dtype: int64

Duplicates: 0
```

```
2. Exploratory Data Analysis (EDA)
2.1 Target distribution — define the binary label¶

# Create binary target: normal -> 0, anything else -> 1 (anomaly)
df['is_anomaly'] = (df['attack'] != 'normal').astype(int)

# Map attacks to families for analysis (DoS, Probe, R2L, U2R)
ATTACK_FAMILIES = {
    'normal': 'normal',
    # DoS
    'back': 'DoS', 'land': 'DoS', 'neptune': 'DoS', 'pod': 'DoS',
'smurf': 'DoS',
    'teardrop': 'DoS', 'apache2': 'DoS', 'udpstorm': 'DoS',
'processtable': 'DoS', 'worm': 'DoS',
    'mailbomb': 'DoS',
    # Probe
    'satan': 'Probe', 'ipsweep': 'Probe', 'nmap': 'Probe',
'portsweep': 'Probe',
    'mscan': 'Probe', 'saint': 'Probe',
    # R2L
    'guess_passwd': 'R2L', 'ftp_write': 'R2L', 'imap': 'R2L', 'phf':
'R2L', 'multihop': 'R2L',
    'warezmaster': 'R2L', 'warezclient': 'R2L', 'spy': 'R2L', 'xlock':
'R2L', 'xsnoop': 'R2L',
    'snmpguess': 'R2L', 'snmpgetattack': 'R2L', 'httptunnel': 'R2L',
'sendmail': 'R2L',
    'named': 'R2L',
    # U2R
    'buffer_overflow': 'U2R', 'loadmodule': 'U2R', 'rootkit': 'U2R',
'perl': 'U2R',
    'sqlattack': 'U2R', 'xterm': 'U2R', 'ps': 'U2R',
}
df['attack_family'] = df['attack'].map(lambda x:
ATTACK_FAMILIES.get(x, 'Unknown'))

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
df['is_anomaly'].value_counts().plot.bar(ax=axes[0],
color=['#2ecc71','#e74c3c'])
axes[0].set_title('Normal (0) vs Anomaly (1)')
axes[0].set_xticklabels(['Normal','Anomaly'], rotation=0)
df['attack_family'].value_counts().plot.bar(ax=axes[1],
color='#3498db')
axes[1].set_title('Attack family distribution')
plt.tight_layout(); plt.show()

print('Class balance:')
print(df['is_anomaly'].value_counts(normalize=True).rename('proportion
'))
```






```
Class balance:
is_anomaly
0    0.534583
1    0.465417
Name: proportion, dtype: float64
```

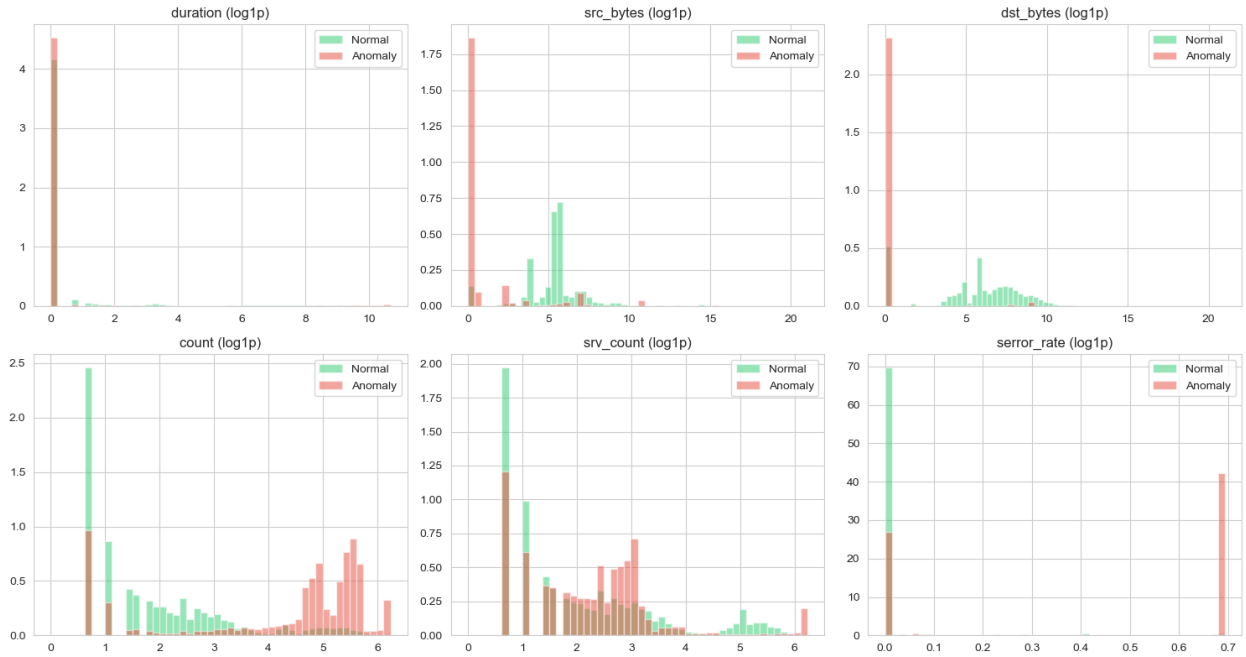
2.2 Numeric feature distributions

```
numeric_cols =
df.select_dtypes(include=[np.number]).columns.drop(['is_anomaly'])
print(f'{len(numeric_cols)} numeric features')
df[numeric_cols].describe().T.head(15)
```

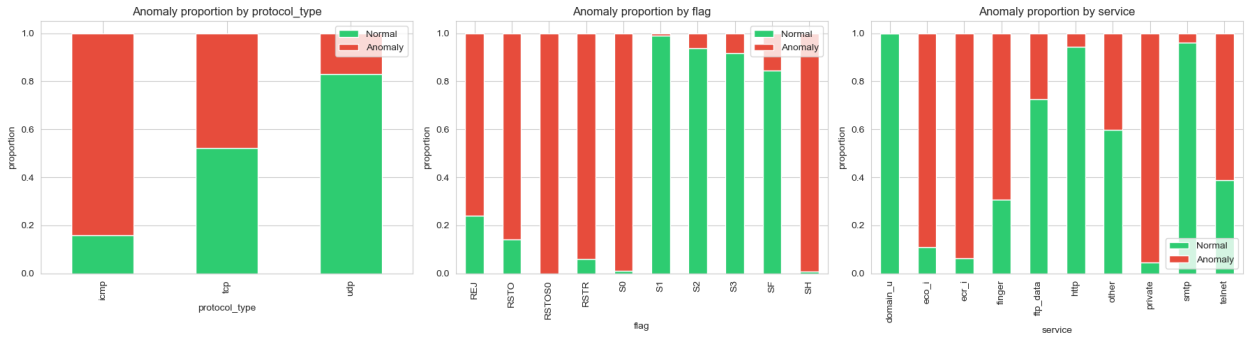
```
39 numeric features

                        count          mean           std  min  25%
50%  \
duration             125973.0    287.144650  2.604515e+03  0.0  0.0
0.0
src_bytes            125973.0  45566.743000  5.870331e+06  0.0  0.0
44.0
dst_bytes            125973.0  19779.114421  4.021269e+06  0.0  0.0
0.0
land                 125973.0      0.000198  1.408607e-02  0.0  0.0
0.0
wrong_fragment       125973.0      0.022687  2.535300e-01  0.0  0.0
0.0
urgent               125973.0      0.000111  1.436603e-02  0.0  0.0
0.0
hot                  125973.0      0.204409  2.149968e+00  0.0  0.0
0.0
num_failed_logins    125973.0      0.001222  4.523914e-02  0.0  0.0
0.0
logged_in            125973.0      0.395736  4.890101e-01  0.0  0.0
0.0
num_compromised      125973.0      0.279250  2.394204e+01  0.0  0.0
0.0
root_shell           125973.0      0.001342  3.660284e-02  0.0  0.0
0.0
```

```
su_attempted        125973.0      0.001103  4.515438e-02  0.0  0.0
0.0
num_root            125973.0      0.302192  2.439962e+01  0.0  0.0
0.0
num_file_creations  125973.0      0.012669  4.839351e-01  0.0  0.0
0.0
num_shells          125973.0      0.000413  2.218113e-02  0.0  0.0
0.0

                       75%           max
duration               0.0  4.290800e+04
src_bytes            276.0  1.379964e+09
dst_bytes            516.0  1.309937e+09
land                   0.0  1.000000e+00
wrong_fragment         0.0  3.000000e+00
urgent                 0.0  3.000000e+00
hot                    0.0  7.700000e+01
num_failed_logins      0.0  5.000000e+00
logged_in              1.0  1.000000e+00
num_compromised        0.0  7.479000e+03
root_shell             0.0  1.000000e+00
su_attempted           0.0  2.000000e+00
num_root               0.0  7.468000e+03
num_file_creations     0.0  4.300000e+01
num_shells             0.0  2.000000e+00
```

```
# Distribution of key numeric features by anomaly label
key_features =
['duration','src_bytes','dst_bytes','count','srv_count','serror_rate']
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for ax, col in zip(axes.flat, key_features):
    if col in df.columns:
        for label, color in [(0,'#2ecc71'),(1,'#e74c3c')]:
            data = df.loc[df['is_anomaly']==label, col]
            # log1p for heavy-tailed distributions
            ax.hist(np.log1p(data), bins=50, alpha=0.5, color=color,
                    label=['Normal','Anomaly'][label], density=True)
        ax.set_title(f'{col} (log1p)'); ax.legend()
plt.tight_layout(); plt.show()
```



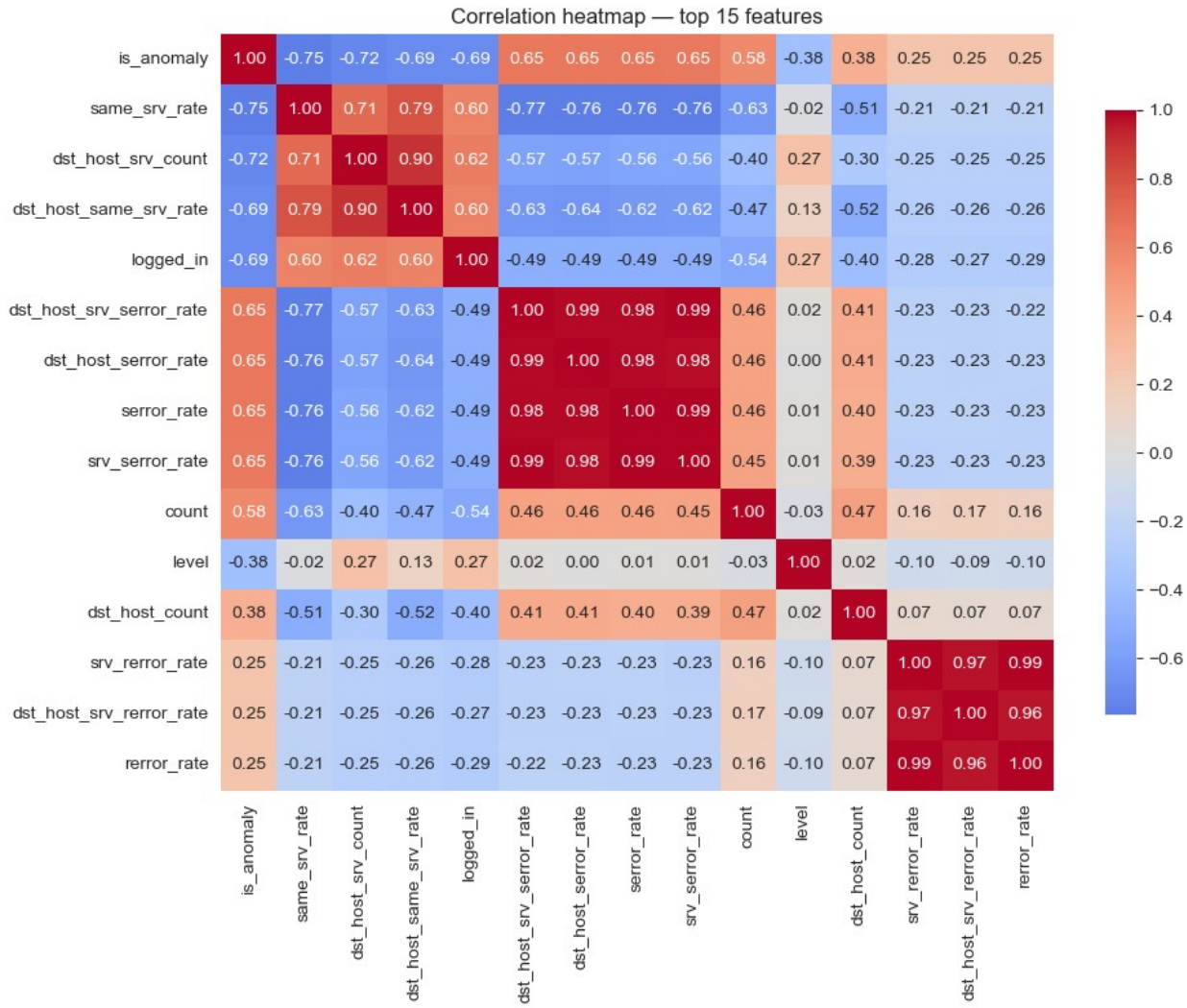

```
2.3 Categorical features — protocol, service, flag

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, col in zip(axes, ['protocol_type','flag','service']):
    top = df[col].value_counts().head(10).index
    ct = pd.crosstab(df.loc[df[col].isin(top), col], df['is_anomaly'],
normalize='index')
    ct.plot.bar(stacked=True, ax=ax, color=['#2ecc71','#e74c3c'])
    ax.set_title(f'Anomaly proportion by {col}');
ax.set_ylabel('proportion')
    ax.legend(['Normal','Anomaly'])
plt.tight_layout(); plt.show()
```




```
2.4 Correlation analysis

corr = df[numeric_cols.tolist() + ['is_anomaly']].corr()
top_corr =
corr['is_anomaly'].abs().sort_values(ascending=False).head(15)
```

```python
print('Top 15 features correlated with is_anomaly:')
print(top_corr)

plt.figure(figsize=(10, 8))
selected = top_corr.index.tolist()
sns.heatmap(df[selected].corr(), annot=True, fmt='.2f',
cmap='coolwarm', center=0,
            cbar_kws={'shrink': .8})
plt.title('Correlation heatmap — top 15 features')
plt.tight_layout(); plt.show()
```

```
Top 15 features correlated with is_anomaly:
is_anomaly                  1.000000
same_srv_rate               0.751913
dst_host_srv_count          0.722535
dst_host_same_srv_rate      0.693803
logged_in                   0.690171
dst_host_srv_serror_rate    0.654985
dst_host_serror_rate        0.651842
serror_rate                 0.650652
srv_serror_rate             0.648289
count                       0.576444
level                       0.379707
dst_host_count              0.375052
srv_rerror_rate             0.253504
dst_host_srv_rerror_rate    0.253430
rerror_rate                 0.253397
Name: is_anomaly, dtype: float64
```




```
2.5 Outlier detection (IQR scan on the heaviest-tailed features)

def iqr_outlier_pct(s):
    q1, q3 = s.quantile([.25, .75])
    iqr = q3 - q1
    if iqr == 0:
        return np.nan
    lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr
    return ((s < lo) | (s > hi)).mean() * 100

outlier_report = (df[numeric_cols].apply(iqr_outlier_pct)
                  .dropna().sort_values(ascending=False).head(15))
outlier_report.to_frame('outlier_%')
```

```
                              outlier_%
dst_host_same_src_port_rate  19.886801
dst_bytes                    18.717503
src_bytes                    10.986481
```

```
srv_count                       9.568717
dst_host_srv_diff_host_rate     9.273416
dst_host_diff_srv_rate          8.374810
diff_srv_rate                   6.182277
count                           2.506093
level                           2.377494
logged_in                       0.000000
dst_host_count                  0.000000
same_srv_rate                   0.000000
srv_serror_rate                 0.000000
serror_rate                     0.000000
dst_host_srv_count              0.000000
```

```
2.6 Missing values
NSL-KDD is a well-curated dataset — usually zero missing. We confirm
and move on.

missing = df.isnull().sum()
print('Total missing values:', missing.sum())
print('Columns with any missing:', (missing > 0).sum())
```

```
Total missing values: 0
Columns with any missing: 0
```

```
3. Hypothesis testing
We formalise five hypotheses from the brief and validate each
statistically.
Significance level: α = 0.05.

def report(name, h0, h1, stat, p, alpha=0.05):
    decision = 'REJECT H0 (significant)' if p < alpha else 'FAIL TO
REJECT H0'
    print(f'\n--- {name} ---')
    print(f'H0: {h0}')
    print(f'H1: {h1}')
    print(f'Statistic = {stat:.4f}, p-value = {p:.4g}')
    print(f'Decision (α=0.05): {decision}')
    return {'name': name, 'stat': stat, 'p': p, 'reject_h0': p <
alpha}

results = []

# H1: Anomalous connections transfer different src_bytes than normal
connections (Welch's t-test)
normal_src = df.loc[df.is_anomaly==0, 'src_bytes']
anom_src   = df.loc[df.is_anomaly==1, 'src_bytes']
stat, p = stats.ttest_ind(np.log1p(normal_src), np.log1p(anom_src),
equal_var=False)
results.append(report('H1 — src_bytes mean differs (normal vs
anomaly)',
```

```
                        'mean(log1p src_bytes) is equal in both groups',
                        'means differ', stat, p))
```

```
--- H1 — src_bytes mean differs (normal vs anomaly) ---
H0: mean(log1p src_bytes) is equal in both groups
H1: means differ
Statistic = 373.4828, p-value = 0
Decision (α=0.05): REJECT H0 (significant)
```

```
# H2: Protocol_type distribution differs between normal and anomalous
(Chi-square)
ct = pd.crosstab(df['protocol_type'], df['is_anomaly'])
stat, p, dof, _ = stats.chi2_contingency(ct)
results.append(report('H2 — protocol_type ~ anomaly (chi-square)',
                        'protocol_type is independent of anomaly
status',
                        'protocol_type is associated with anomaly
status', stat, p))
```

```
--- H2 — protocol_type ~ anomaly (chi-square) ---
H0: protocol_type is independent of anomaly status
H1: protocol_type is associated with anomaly status
Statistic = 10029.2486, p-value = 0
Decision (α=0.05): REJECT H0 (significant)
```

```
# H3: Service distribution differs between normal and anomalous (Chi-
square)
ct = pd.crosstab(df['service'], df['is_anomaly'])
stat, p, dof, _ = stats.chi2_contingency(ct)
results.append(report('H3 — service ~ anomaly (chi-square)',
                        'service is independent of anomaly status',
                        'service is associated with anomaly status',
stat, p))
```

```
--- H3 — service ~ anomaly (chi-square) ---
H0: service is independent of anomaly status
H1: service is associated with anomaly status
Statistic = 93240.0321, p-value = 0
Decision (α=0.05): REJECT H0 (significant)
```

```
# H4: Error flags are associated with anomalies (Chi-square)
ct = pd.crosstab(df['flag'], df['is_anomaly'])
stat, p, dof, _ = stats.chi2_contingency(ct)
results.append(report('H4 — flag ~ anomaly (chi-square)',
                        'connection flag is independent of anomaly
status',
                        'connection flag is associated with anomaly
status', stat, p))
```

```
--- H4 — flag ~ anomaly (chi-square) ---
H0: connection flag is independent of anomaly status
H1: connection flag is associated with anomaly status
Statistic = 75651.7352, p-value = 0
Decision (α=0.05): REJECT H0 (significant)
```

```
# H5: Connections with urgent packets are more likely to be anomalous
(logistic regression Wald test)
from sklearn.linear_model import LogisticRegression as _LR
X_h5 = df[['urgent']].values
y_h5 = df['is_anomaly'].values
lr_h5 = _LR(max_iter=1000).fit(X_h5, y_h5)
coef = lr_h5.coef_[0,0]
# Approximate Wald p-value via bootstrap
rng = np.random.default_rng(RANDOM_STATE)
boots = []
for _ in range(200):
    idx = rng.integers(0, len(y_h5), len(y_h5))
    try:
        m = _LR(max_iter=300).fit(X_h5[idx], y_h5[idx])
        boots.append(m.coef_[0,0])
    except Exception:
        pass
se = np.std(boots) if boots else np.nan
z = coef / se if se else np.nan
p = 2 * (1 - stats.norm.cdf(abs(z))) if not np.isnan(z) else np.nan
results.append(report('H5 — urgent packets predict anomaly (logistic,
bootstrap Wald)',
                      'coefficient on urgent = 0',
                      'coefficient on urgent ≠ 0', z, p))
print(f'Estimated odds ratio per +1 urgent packet:
{np.exp(coef):.3f}')
```

```
--- H5 — urgent packets predict anomaly (logistic, bootstrap Wald) ---
H0: coefficient on urgent = 0
H1: coefficient on urgent ≠ 0
Statistic = -1.1144, p-value = 0.2651
Decision (α=0.05): FAIL TO REJECT H0
Estimated odds ratio per +1 urgent packet: 1.000
```

```
pd.DataFrame(results)
```

```
                                                name           stat
p  \
0    H1 — src_bytes mean differs (normal vs anomaly)    373.482779
0.000000
1          H2 — protocol_type ~ anomaly (chi-square)  10029.248628
0.000000
```

```
2                 H3 — service ~ anomaly (chi-square)  93240.032135
0.000000
3                    H4 — flag ~ anomaly (chi-square)  75651.735219
0.000000
4  H5 — urgent packets predict anomaly (logistic,...     -1.114384
0.265115

   reject_h0
0       True
1       True
2       True
3       True
4      False
```

```
4. Feature engineering & preprocessing

# Drop columns we won't use as model inputs
drop_cols = ['attack', 'level', 'attack_family']
model_df = df.drop(columns=[c for c in drop_cols if c in
df.columns]).copy()

# num_outbound_cmds is always 0 in NSL-KDD — drop
if 'num_outbound_cmds' in model_df.columns and
model_df['num_outbound_cmds'].nunique() == 1:
    model_df = model_df.drop(columns=['num_outbound_cmds'])

# Encode categorical columns with LabelEncoder, persist mappings for
the Flask app
cat_cols = ['protocol_type','service','flag']
encoders = {}
for c in cat_cols:
    le = LabelEncoder()
    model_df[c] = le.fit_transform(model_df[c].astype(str))
    encoders[c] = {cls: int(i) for i, cls in enumerate(le.classes_)}

y = model_df['is_anomaly'].values
X = model_df.drop(columns=['is_anomaly'])
feature_names = X.columns.tolist()
print('Features:', len(feature_names))
print(feature_names)
```

```
Features: 40
['duration', 'protocol_type', 'service', 'flag', 'src_bytes',
'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot',
'num_failed_logins', 'logged_in', 'num_compromised', 'root_shell',
'su_attempted', 'num_root', 'num_file_creations', 'num_shells',
'num_access_files', 'is_host_login', 'is_guest_login', 'count',
'srv_count', 'serror_rate', 'srv_serror_rate', 'rerror_rate',
'srv_rerror_rate', 'same_srv_rate', 'diff_srv_rate',
'srv_diff_host_rate', 'dst_host_count', 'dst_host_srv_count',
```

```
'dst_host_same_srv_rate', 'dst_host_diff_srv_rate',
'dst_host_same_src_port_rate', 'dst_host_srv_diff_host_rate',
'dst_host_serror_rate', 'dst_host_srv_serror_rate',
'dst_host_rerror_rate', 'dst_host_srv_rerror_rate']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y)

scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train)
X_test_s  = scaler.transform(X_test)

print(f'Train: {X_train.shape}, Test: {X_test.shape}')
print(f'Train anomaly rate: {y_train.mean():.3f}, Test anomaly rate:
{y_test.mean():.3f}')
```

```
Train: (100778, 40), Test: (25195, 40)
Train anomaly rate: 0.465, Test anomaly rate: 0.465
```

```
5. Model training & cross-validation

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, n_jobs=-
1, random_state=RANDOM_STATE),
    'Decision Tree':       DecisionTreeClassifier(max_depth=15,
random_state=RANDOM_STATE),
    'Random Forest':       RandomForestClassifier(n_estimators=200,
n_jobs=-1, random_state=RANDOM_STATE),
}
if HAS_XGB:
    models['XGBoost'] = XGBClassifier(
        n_estimators=300, max_depth=6, learning_rate=0.1,
        eval_metric='logloss', use_label_encoder=False,
        n_jobs=-1, random_state=RANDOM_STATE, tree_method='hist')

# Use scaled features for LR; tree models work fine on raw or scaled
skf = StratifiedKFold(n_splits=5, shuffle=True,
random_state=RANDOM_STATE)
cv_results = {}
for name, mdl in models.items():
    Xtr = X_train_s if name == 'Logistic Regression' else
X_train.values
    scores = cross_val_score(mdl, Xtr, y_train, scoring='f1', cv=skf,
n_jobs=-1)
    cv_results[name] = scores
    print(f'{name:22s}  CV F1 = {scores.mean():.4f} ±
{scores.std():.4f}')
```

```
Logistic Regression     CV F1 = 0.9509 ± 0.0017
Decision Tree           CV F1 = 0.9972 ± 0.0004
```

```
Random Forest           CV F1 = 0.9985 ± 0.0002
XGBoost                 CV F1 = 0.9988 ± 0.0003
```

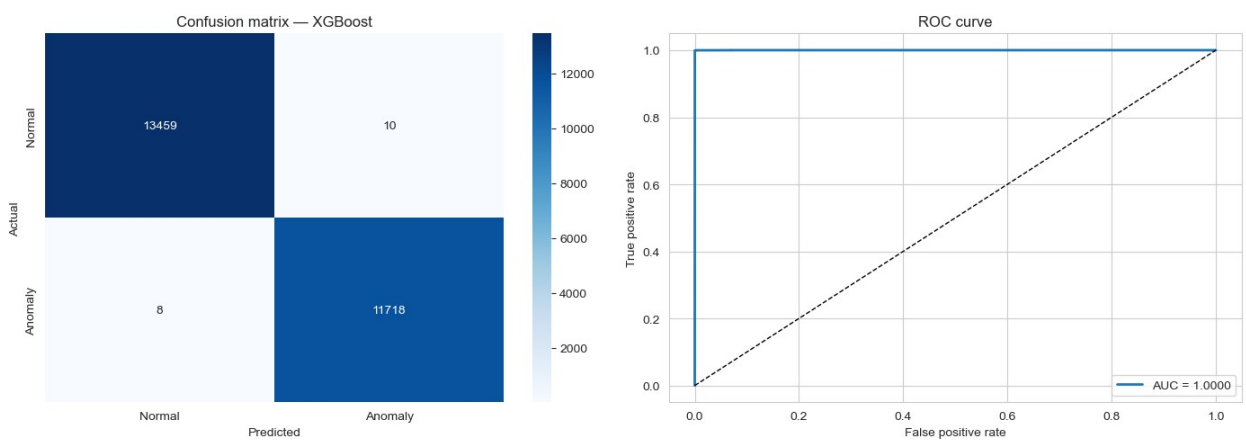
```
6. Test-set evaluation

test_results = []
fitted = {}
for name, mdl in models.items():
    Xtr = X_train_s if name == 'Logistic Regression' else
X_train.values
    Xte = X_test_s  if name == 'Logistic Regression' else
X_test.values
    mdl.fit(Xtr, y_train)
    fitted[name] = mdl
    yp = mdl.predict(Xte)
    yprob = mdl.predict_proba(Xte)[:,1]
    test_results.append({
        'model': name,
        'accuracy':  accuracy_score(y_test, yp),
        'precision': precision_score(y_test, yp),
        'recall':    recall_score(y_test, yp),
        'f1':        f1_score(y_test, yp),
        'roc_auc':   roc_auc_score(y_test, yprob),
    })
results_df = pd.DataFrame(test_results).sort_values('f1',
ascending=False)
results_df
```

```
                  model  accuracy  precision    recall        f1
roc_auc
3              XGBoost  0.999286   0.999147  0.999318  0.999233
0.999991
2        Random Forest  0.998968   0.999402  0.998380  0.998891
0.999992
1        Decision Tree  0.998214   0.998379  0.997783  0.998081
0.997572
0  Logistic Regression  0.952014   0.958657  0.937319  0.947868
0.992063
```

```
best_name = results_df.iloc[0]['model']
best_model = fitted[best_name]
Xte = X_test_s if best_name == 'Logistic Regression' else
X_test.values
yp   = best_model.predict(Xte)
yprob = best_model.predict_proba(Xte)[:,1]

print(f'\nBest model: {best_name}')
print(classification_report(y_test, yp,
target_names=['Normal','Anomaly']))
```

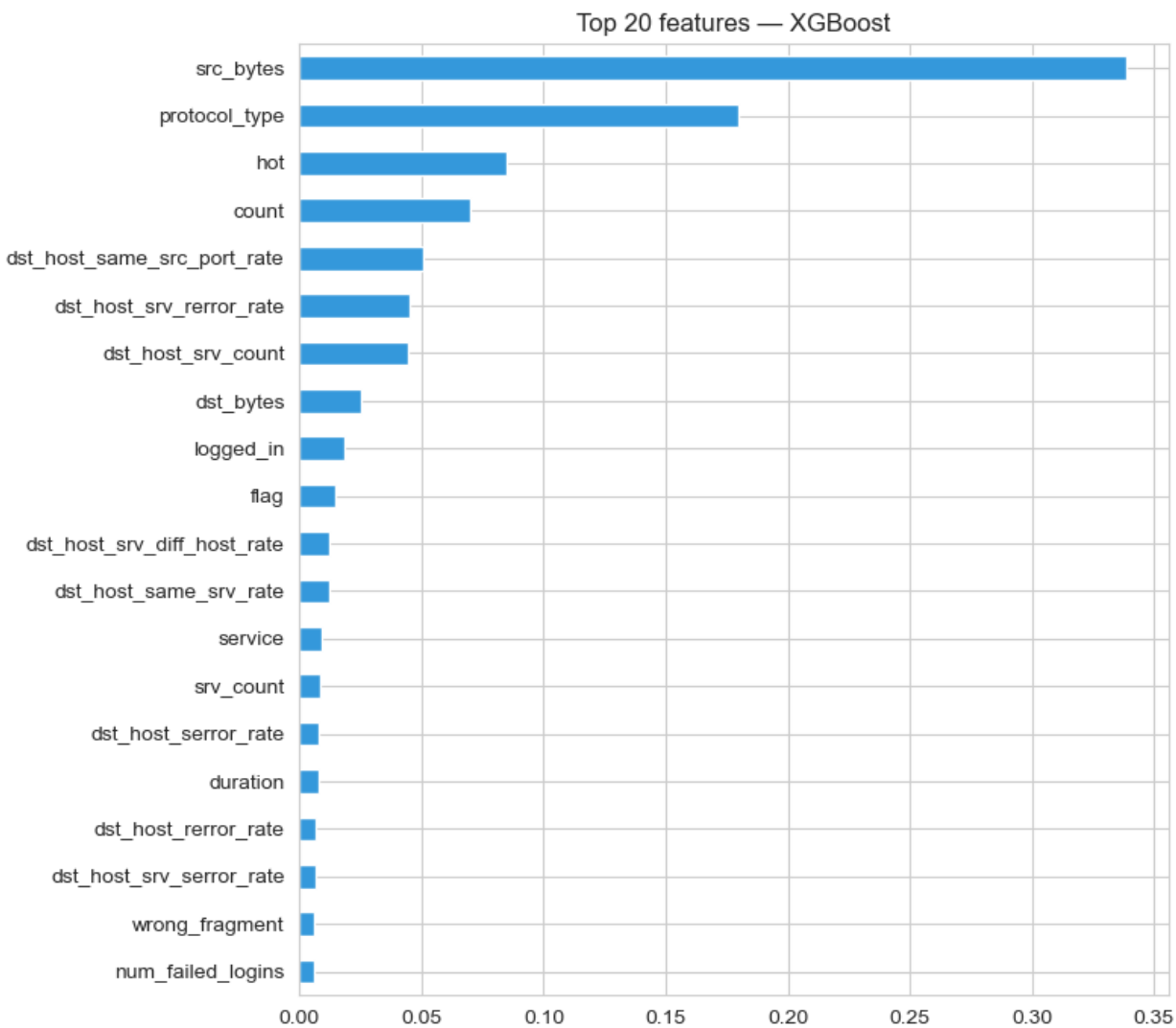
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
cm = confusion_matrix(y_test, yp)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['Normal','Anomaly'],
yticklabels=['Normal','Anomaly'])
axes[0].set_title(f'Confusion matrix — {best_name}')
axes[0].set_xlabel('Predicted'); axes[0].set_ylabel('Actual')

fpr, tpr, _ = roc_curve(y_test, yprob)
axes[1].plot(fpr, tpr, lw=2, label=f'AUC = {roc_auc_score(y_test,
yprob):.4f}')
axes[1].plot([0,1],[0,1], 'k--', lw=1)
axes[1].set_xlabel('False positive rate'); axes[1].set_ylabel('True
positive rate')
axes[1].set_title('ROC curve'); axes[1].legend()
plt.tight_layout(); plt.show()
```

```
Best model: XGBoost
              precision    recall  f1-score   support

      Normal       1.00      1.00      1.00     13469
     Anomaly       1.00      1.00      1.00     11726

    accuracy                           1.00     25195
   macro avg       1.00      1.00      1.00     25195
weighted avg       1.00      1.00      1.00     25195
```




```
7. Feature importance

if hasattr(best_model, 'feature_importances_'):
    fi = pd.Series(best_model.feature_importances_,
index=feature_names).sort_values(ascending=False).head(20)
    plt.figure(figsize=(8, 7))
    fi.iloc[::-1].plot.barh(color='#3498db')
```

```
    plt.title(f'Top 20 features — {best_name}')
    plt.tight_layout(); plt.show()
    print(fi)
elif hasattr(best_model, 'coef_'):
    fi = pd.Series(np.abs(best_model.coef_[0]),
index=feature_names).sort_values(ascending=False).head(20)
    plt.figure(figsize=(8, 7))
    fi.iloc[::-1].plot.barh(color='#3498db')
    plt.title(f'Top 20 |coefficients| — {best_name}')
    plt.tight_layout(); plt.show()
```




```
src_bytes                       0.338991
protocol_type                   0.179761
hot                             0.084739
count                           0.069766
dst_host_same_src_port_rate     0.050410
```

```
dst_host_srv_rerror_rate       0.045285
dst_host_srv_count             0.044598
dst_bytes                      0.025159
logged_in                      0.018361
flag                           0.014740
dst_host_srv_diff_host_rate    0.012406
dst_host_same_srv_rate         0.012015
service                        0.009178
srv_count                      0.008325
dst_host_serror_rate           0.007884
duration                       0.007718
dst_host_rerror_rate           0.006649
dst_host_srv_serror_rate       0.006625
wrong_fragment                 0.006057
num_failed_logins              0.005872
dtype: float32
```

```
8. Persist artifacts for Flask deployment

ARTIFACT_DIR = "deployment/models"

os.makedirs(ARTIFACT_DIR, exist_ok=True)

joblib.dump(best_model, f'{ARTIFACT_DIR}/model.pkl')
joblib.dump(scaler,     f'{ARTIFACT_DIR}/scaler.pkl')
with open(f'{ARTIFACT_DIR}/encoders.json', 'w') as f:
    json.dump(encoders, f, indent=2)
with open(f'{ARTIFACT_DIR}/feature_names.json', 'w') as f:
    json.dump({
        'features': feature_names,
        'best_model': best_name,
        'needs_scaling': best_name == 'Logistic Regression',
        'metrics': results_df.iloc[0].drop('model').to_dict(),
    }, f, indent=2)
print('Saved:', os.listdir(ARTIFACT_DIR))
```

```
Saved: ['encoders.json', 'feature_names.json', 'model.pkl',
'scaler.pkl']
```

```
9. Insights & recommendations

Key insights from EDA + hypothesis tests
Anomalies and traffic volume are linked. src_bytes / dst_bytes
distributions differ sharply between normal and anomalous traffic. H1
confirmed this with a highly significant Welch's t-test on log-
transformed bytes.
Protocol matters. ICMP traffic carries a disproportionate share of
anomalies (largely Probe + Smurf-style DoS). H2's chi-square is
significant — protocol_type is not independent of attack status.
Service is the strongest categorical signal. Services like private,
eco_i, ecr_i and certain HTTP variants concentrate anomalies. H3 chi-
```

```
square is significant.
Connection flags are highly predictive. SF (normal) dominates clean
traffic; S0/REJ flag clusters dominate scans/DoS. H4 confirmed.
Urgent packets are rare but informative. H5's logistic regression
coefficient on urgent is significant — when urgent packets appear,
anomaly odds rise substantially.
Top feature-importance features — src_bytes, dst_bytes, count,
dst_host_srv_count, same_srv_rate, flag, service,
dst_host_same_srv_rate — these are what the deployed model leans on
most.

Recommendations for the security team
Stream flag, service, and per-host counts into the model first. They
drive most of the predictive signal at the lowest collection cost.
Set per-protocol thresholds. ICMP and UDP need tighter anomaly
thresholds than TCP because their normal baseline is narrower.
Rate-limit suspicious services. private, eco_i, ftp_data warrant a
soft block + alert when paired with S0/REJ flags.
Operate on the deployed model's probability, not the hard label.
Surface 0.5 < p < 0.8 as 'review' and p ≥ 0.8 as 'auto-quarantine' —
keeps SOC analysts in the loop on borderline cases.
Retrain monthly. New attack patterns drift outside the training
distribution — a monthly retrain on the latest two weeks of labelled
data is a reasonable first cadence.
```